

Windows-Linux-Integration

Allgemeines

Eine Vielzahl von Unternehmen nutzen für ihre Benutzerverwaltung und Authentifizierung innerhalb der firmeneigenen Netzwerke Windows Active Directory (AD). AD basiert auf Verzeichnissen. Verzeichnisse in Bezug auf Netzwerke sind Strukturen zur Ablage von Informationen und Objekten. Das AD bietet mit seinen Active Directory Domain Services (AD DS) die Möglichkeit, Benutzerkonten, Server, Netzwerkfreigaben oder Computer zu speichern, auf die andere autorisierte Benutzer zugreifen können. Informationen, die zu den Benutzern gespeichert werden, sind zum Beispiel: (vgl. [ACT1])

- Name
- Kennwörter
- Telefonnummern

Alle diese gespeicherten Informationen werden den entsprechenden Objekten zugeordnet. Die Administratoren eines Unternehmens haben somit die Möglichkeit, dass entsprechende Ressourcen ohne Probleme gefunden, verwaltet und genutzt werden können. (vgl. [ACT1])

Ein wesentliches Sicherheitsfeature ist dabei die Anmeldeauthentifizierung und Steuerung des Zugriffs. Mit einer Anmeldung im Netzwerk wird verwaltet, welcher Benutzer auf welche Ressourcen innerhalb des Netzwerks Zugriff hat. (vgl. [ACT1])

Generell gesehen erlaubt das Active Directory, dass ein reales Unternehmen samt seiner Mitarbeiterinnen und Mitarbeitern, den einzelnen Abteilungen und genutzten Geräte wie Drucker und Scanner abgebildet wird. Alle Informationen werden dementsprechend organisiert, gegliedert und auch überwacht. (vgl. [ACT2])

Kerberos

Kerberos ist ein spezielles Protokoll, welches in den 80er-Jahren entwickelt wurde und für die Authentifizierung zwischen mindestens zwei verschiedenen Hosts eingesetzt wird. Die Kerberos Authentifizierung wurde mit Windows 2000 eingeführt und ist mittlerweile der Standard für Authentifizierung innerhalb von Microsoft Windows. Der Name geht zurück auf den dreiköpfigen Höllenhund Kerberos, der in der griechischen Mythologie den Eingang zur Unterwelt beschützt und sichert. (vgl. [KER1])

Die Teilnehmer an der Authentifizierung

Client

Der Client ist der Rechner, der eine Anfrage an einen entsprechenden Dienst stellt, bei dem man sich authentifizieren muss. Generell stößt somit der Client die eigentliche Kommunikation an. (vgl. [KER1])

Hosting Server

Der Host ist derjenige Server, auf dem sich der Dienst befindet, auf den der Client zugreifen möchte. Gegenüber diesem Server muss sich der Client folglich authentifizieren. (vgl. [KER1])

Authentifizierungsserver

Hierbei handelt es sich um den Server, welche die eigentliche Authentifizierung durchführt, damit der Client auf den Dienst innerhalb des Hosting Servers zugreifen kann bzw. darf. (vgl. [KER1])

Ticket Granting Server

Über diesen Server werden Service Tickets (ST) an den Client ausgestellt. (vgl. [KER1])

Key Distribution Center

Das Key Distribution Center ist der Oberbegriff für den Authentifizierungsserver und den Ticket Granting Server. (vgl. [KER1])

Der Authentifizierungsprozess

Der eigentliche Authentifizierungsprozess mit Hilfe von Kerberos kann in 6 Schritte untergliedert werden:

Schritt 1

Der Client sendet eine verschlüsselte Anfrage an den Authentifizierungsserver (AS). Der Authentifizierungsserver überprüft die Anfrage, indem er mit der ID des Benutzers das entsprechende Passwort findet und damit die Anfrage entschlüsselt. Nur wenn der Client die korrekten Daten verwendet, kann der Authentifizierungsserver die Anfrage erfolgreich entschlüsseln und die Identität des Benutzers bestätigen. (vgl. [KER1])

Schritt 2

Sofern die Authentifizierung erfolgreich war, sendet das Key Distribution Center (KDC), in diesem Fall der Authentifizierungsserver, ein Ticket Granting Ticket (TGT) an den Client zurück. Das TGT ist verschlüsselt und kann nur vom Ticket Granting Server (TGS) entschlüsselt werden. (vgl. [KER1])

Schritt 3

Der Client speichert das TGT ab und verwendet es, um sich beim Ticket Granting Server (TGS) zu authentifizieren. Der Client schickt das TGT zusammen mit einer Serviceanfrage an den TGS. (vgl. [KER1])

Schritt 4

Der TGS überprüft das TGT mit seinem geheimen Schlüssel. Wenn das TGT gültig ist, stellt der TGS ein Service Ticket für den Client aus. (vgl. [KER1])

Schritt 5

Der Client speichert das Service Ticket und schickt es an den entsprechenden Hosting Server (z. B. einen Anwendungsserver), den er benötigt. Der Hosting Server überprüft das Service Ticket, indem er es mit seinem geheimen Schlüssel entschlüsselt. (vgl. [KER1])

Schritt 6

Stimmt der Schlüssel, wird dem Client für die vereinbarte Zeit innerhalb des Service Tickets der Zugriff auf den Dienst gewährt. Der Hosting Server akzeptiert das Service Ticket und gewährt dem Client somit den angeforderten Zugriff. (vgl. [KER1])

klist

Der Befehl `klist` liefert ohne weitere Parameter eine Liste aller vorhandenen Kerberos Tickets, die für den entsprechend angemeldeten Benutzer ausgestellt wurden. Ein beispielhaftes Ticket könnte folgendermaßen aussehen: (vgl. [KLI1])

```
Client: testuser123 @ TEST-DOMAIN.DE
Server: krbtgt/TEST-DOMAIN.DE @ TEST-DOMAIN.DE
KerbTicket (Verschlüsselungstyp): AES-256-CTS-HMAC-SHA1-96
Ticketkennzeichen 0x60a10000 -> forwardable renewable pre_authent name_canonicalize
Startzeit: 6/21/2024 7:40:36 (lokal)
Endzeit: 6/21/2024 17:40:34 (lokal)
Erneuerungszeit: 6/21/2024 17:40:34 (lokal)
Sitzungsschlüsseltyp: AES-256-CTS-HMAC-SHA1-96
Cachekennzeichen: 0x2 -> DELEGATION
KDC aufgerufen: sw2k22vRdc01.test-domain.de
```

Die einzelnen Objekte benötigen allerdings wieder etwas Erläuterung:

- **Client :**
Hinter dem Client verbirgt sich der User, der das entsprechende Ticket angefordert hat. Neben der ID des Users wird hier zusätzlich die Domäne angegeben.
- **Server :**
Der Server ist das eigentliche Key Distribution Center (KDC), welches für die Ausstellung von Tickets verantwortlich ist. Im Beispiel heißt der entsprechende Rechner `krbtgt` und befindet sich ebenfalls in der gleichen Domäne wie der Ticketanforderer.
- **KerbTicket (Verschlüsselungstyp) :**
Mit dieser Eigenschaft wird die Verschlüsselung für das Ticket spezifiziert. Je nach Art und Stärke der Verschlüsselung stehen hier unterschiedliche Werte.
- **Ticketkennzeichen :**
Das Ticketkennzeichen ist ein Hexadezimalwert, womit das Ticket identifiziert werden kann. Neben der Kennzeichnung werden noch weitere relevante Informationen hier abgelegt:
 - **forwarded :**
Hierüber wird spezifiziert, dass das Ticket an andere Systeme weitergeleitet werden kann bzw. darf.
 - **renewable :**
Jedes Ticket kann natürlich auch ablaufen. Mit der Eigenschaft wird gekennzeichnet, ob das Ticket erneuert werden darf, oder nicht.
 - **pre_authent :**
Mit diesem Flag wird angegeben, dass sich der Benutzer vor der Ausstellung des Tickets erfolgreich authentifiziert hat.
 - **name_canonicalize :** Kanonisieren bedeutet, dass etwas standardisiert wurde, also in eine einheitliche Form gebracht wird. Benutzer und Dienste werden folglich in eine einheitliche Form der Domäne gebracht.
- **Startzeit :**
Mit der Startzeit wird das Datum und die Uhrzeit spezifiziert, ab dem das Ticket gültig wurde.
- **Endzeit :**
Datum und Uhrzeit, ab dem das Ticket ungültig wird.
- **Erneuerungszeit :**
Datum und Uhrzeit, bis zu dem das Ticket erneuert werden kann.
- **Sitzungsschlüsseltyp :**
Zwischen Client und Server findet eine verschlüsselte Kommunikation statt. Die Eigenschaft spezifiziert in diesem Zusammenhang den Verschlüsselungstyp für den Sitzungsschlüssel.

- Cachekennzeichen :
Das Cachekennzeichen in Kombination mit der Eigenschaft `DELEGATION` ist ein Flag, das anzeigt, dass das Ticket für die Delegation verwendet werden kann. Delegation erlaubt es einem Dienst, im Namen des Benutzers auf Ressourcen zuzugreifen.
- KDC aufgerufen :
Dies ist der Hostname des Key Distribution Centers (KDC), das das Ticket ausgestellt hat.

Kerberos unter Linux

Installation

Um Kerberos unter Linux überhaupt installieren zu können, sind zwei Pakete relevant, welche installiert werden müssen:

```
apt install libpam-krb5 krb5-user
```

TERMINAL

Während der Installation wird aufgefordert, dass etwaige Konfigurationen durchgeführt werden. Hier einfach nichts eintragen.

Die Testumgebung

Innerhalb der EDV-Schulen gibt es einen eigenen Domänencontroller für die Klassen B11a und B11b:

```
#B11a
10.0.142.1

#B11b
10.0.142.2
```

TERMINAL

Auf jedem dieser Domänencontroller existiert eine eigene Domäne:

```
#B11a
b11a.local

#B11b
b11b.local
```

TERMINAL

Die Domänencontroller sind über nachfolgende Namen erreichbar:

```
#B11a
b11a.dvs-plattling.de

#B11b
b11b.dvs-plattling.de
```

TERMINAL

nslookup

Bei `nslookup` handelt es sich um ein spezielles Tooling, worüber DNS Abfragen durchgeführt werden können. Es wird ermöglicht, dass Benutzer zu einem bestimmten Host bzw. zu einer IP entsprechende Informationen des DNS abfragen können. Für die Verwendung gibt es mehrere Möglichkeiten:

Auflösung eines Hostnamen zu einer IP

Durch die Eingabe eines Hostnamen beim Befehl `nslookup` wird die entsprechende dahinterliegende IP ausgegeben:

```
nslookup b11b.dvs-plattling.de
```

Als Ergebnis erhält zum Beispiel die nachfolgende Ausgabe:

```
Server:      10.0.104.54
Address:     10.0.104.54#53

Name:   b11b.dvs-plattling.de
Address: 10.0.142.2
```

Neben der IP des Hosts erhält man auch den zuständigen DNS Server, der für die Namensauflösung zuständig ist.

Auflösung einer IP zu einem Hostnamen

Durch die Eingabe einer IP Adresse erhält man den entsprechenden Namen des Hosts. Es handelt sich hierbei um eine Reverse DNS Namensauflösung:

```
nslookup 10.0.142.2
```

Das Ergebnis sieht folgendermaßen aus:

```
Server:      10.0.104.54
Address:     10.0.104.54#53

2.142.0.10.in-addr.arpa name = b11b.dvs-plattling.de.
```

Konfiguration von Kerberos

Sobald die Installation abgeschlossen ist, kann man sich an die Grundkonfiguration von Kerberos wagen. Unter `/etc/krb5.conf` ist die entsprechende Konfigurationsdatei abgespeichert. Die Datei beinhaltet bereits eine erste Grundkonfiguration, welche allerdings angepasst werden muss. (vgl. [KER2])

[libdefaults]

Der Bereich `[libdefaults]` beinhaltet grundlegende Konfigurationseinstellungen für Kerberos. Einige mögliche Einstellungen werden im Folgenden erläutert: (vgl. [KER2])

- `default_realm` :
Ein Realm ist mehr oder weniger ein logisches Netzwerk, ähnlich zu einer Domäne. Zu ihm gehören praktisch mehrere Systeme, die von einem KDC verwaltet werden. Mit Hilfe des `default_realm` wird in Großbuchstaben der Name eines Realms vergeben! Ein Beispiel dafür könnte sein `SCHUELER.ORG` oder `B11B.LOCAL` . (vgl. [KER2])
- `ccache_type` :
Der Credential Cache ist ein Mechanismus für Caching innerhalb von Kerberos, welcher die entsprechenden Benutzerdaten beinhaltet, solange diese noch gültig sind. Im Endeffekt kann es angesehen werden wie eine Art Session, damit nicht jedes mal erneut bei einer Authentifizierung das KDC kontaktiert werden muss. Die Zahl gibt dabei die Art des Credential Caches an. `4` bedeutet z. B. das Tickets in einem Verzeichnis gespeichert werden. Jedes Ticket entspricht somit einer eigenen Datei. Standardmäßig steht der Wert auf `4` . (vgl. [KER2])
- `forwardable` :
Hierbei handelt es sich um eine Ticketweiterleitung. Benutzer, die bereits auf einem Host authentifiziert worden sind, können ohne erneute Authentifizierung Tickets an einen anderen Host weiterleiten. Besonders sinnvoll ist das,

wenn Benutzer von einem System auf ein anderes System durchgreifen müssen bzw. wollen, ohne sich immer neu authentifizieren zu müssen. Es wird somit ein Single Sign-On über mehrere Systeme hinweg ermöglicht. Im Standard ist diese Eigenschaft auf `false` . (vgl. [KER2])

- `proxiable` :
Ein Proxiable Ticket wird hauptsächlich dann verwendet, damit ein anderer Dienst im Namen des ursprünglichen Benutzers handeln darf, ohne dass der Benutzer direkt involviert ist. Im Standard ist diese Eigenschaft auf `false` gesetzt. (vgl. [KER2])
- `rdns` :
`rdns` steht für Reverse DNS und ist dafür verantwortlich, dass anhand einer IP Adresse ein entsprechender Hostname gefunden wird. Folglich das genau Gegenteil zum herkömmlichen DNS. Im Standard steht dieser Wert auf `true` . (vgl. [KER2])
- `fcc-mit-ticketflags` :
`fcc` steht für File Credential Cache. Mit der Einstellung wird spezifiziert, ob bestimmte Ticketflags im Credential Cache gespeichert werden, oder nicht. (vgl. [KER2])
- `ticket_lifetime` :
Mit dieser Eigenschaft wird die Gültigkeitsdauer des Tickets spezifiziert. Die Dauer definiert die maximale Lebensdauer eines Kerberos-Tickets, bevor es abläuft und erneuert werden muss. Die Angabe erfolgt in Sekunden. Im Standard liegt die Gültigkeitsdauer bei einem Tag, also 86400 Sekunden. (vgl. [KER2])
- `clockskew` :
Hierrüber wird die maximale Differenz in Sekunden zwischen dem Client und dem KDC angegeben. Diese Differenz zeigt die Toleranz an, die genutzt wird, um ein Ticket als gültig zu akzeptieren. Kerberos selbst basiert sehr stark auf Timestamps, damit sichergestellt werden kann, dass Tickets auch nur für eine begrenzte Zeit gültig sind. Uhren auf Systemen können unterschiedlich sein. `clockskew` erlaubt somit eine gewisse Zeitabweichung. Im Standard liegt der Wert bei 300 , also 5 Minuten. (vgl. [KER2])
- `dns_fallback` :
Sollte aus einem speziellen Grund die DNS Namensauflösung des KDC oder des Realms fehlschlagen, so kann eine alternative Methode zur Auflösung verwendet werden. Somit wird sichergestellt, dass ein Client das KDC bzw. den Realm trotzdem erfolgreich auflösen kann, auch wenn die primäre DNS Namensauflösung nicht klappen sollte. (vgl. [KER2])
- `dns_lookup_realm` :
Mit Hilfe dieser Einstellung versucht der Kerberos-Client, den Namen des Realms anhand des DNS Domänennamens zu ermitteln. Dies bedeutet, dass der Client versucht, den Namen des Realms basierend auf dem Namen der Domäne zu bestimmen, ohne dass der Realm explizit konfiguriert werden muss. Im Standard ist diese Eigenschaft auf `false` . (vgl. [KER2])
- `dns_lookup_kdc` :
Ähnlich wie schon `dns_lookup_realm` wird hier versucht den Namen des KDC anhand des DNS Domänennamens zu bestimmen. Im Standard ist diese Eigenschaft auf `true` . (vgl. [KER2])

Eine beispielhafte Konfiguration für den Bereich `[libdefaults]` sieht folgendermaßen aus:

```
[libdefaults]
    default_realm = <NAME_OF_REALM>
    ccache_type = 4
    forwardable = true
    proxiable = true
    rdns = false
    fcc-mit-ticketflags = true
    ticket_lifetime = 24000
    clockskew = 600
    dns_fallback = on
    dns_lookup_realm = false
    dns_lookup_kdc = false
```

[realms]

Innerhalb dieses Konfigurationsabschnitts wird angegeben, wie die spezifische Konfiguration für einen ganz bestimmten Realm aussieht. Dabei sind die nachfolgenden Eigenschaften relevant: (vgl. [KER2])

- **kdc :**
Über diese Eigenschaft wird die Adresse des Key Distribution Centers angegeben. Hierfür wird der Port 88 verwendet. Das KDC ist hauptsächlich für die Authentifizierung und die entsprechende Ticketvergabe zuständig. (vgl. [KER2])
- **admin_server :**
Hier wird die Adresse für den Administrationsserver eingetragen, welcher verantwortlich ist für die Verwaltung des Realms und den entsprechenden administrativen Aufgaben. Als Port wird hier der 464 genutzt. (vgl. [KER2])
- **default_domain :**
Die `default_domain` Einstellung definiert die Domäne, die verwendet wird, wenn ein Benutzer sich ohne explizite Angabe einer Domäne anmeldet. Diese Einstellung ist insbesondere dann relevant, wenn Benutzer sich mit ihrem Benutzernamen anmelden, ohne die Domäne zu spezifizieren. (vgl. [KER2])

Eine mögliche Konfiguration für den Breich `[realms]` könnte folgendermaßen aussehen:

```
[realms]
    <NAME_OF_REALM> = {
        kdc = <IP_of_DC>:88
        admin_server = <IP_of_DC>:464
        default_domain = <NAME_OF_REALM>
    }
```

[domain_realm]

Mit Hilfe dieser Einstellung wird eine Zuordnung zwischen Domänen und Realms durchgeführt. Dies ermöglicht es Kerberos, den richtigen Realm für einen bestimmten Host oder eine Domäne zu bestimmen, wenn eine Authentifizierungsanfrage eingeht. Vom Aufbau sieht die Konfiguration folgendermaßen aus. (vgl. [KER2])

```
[domain_realm]
    <Name_of_Domain> = <NAME_OF_REALM>
```

Vollständige Konfiguration

```
[libdefaults]
    default_realm = B11B.LOCAL
    forwardable = true
    proxiable = true
    rdns = false
    ticket_lifetime = 24000
    clocks skew = 600
    dns_lookup_kdc = false

[realms]
    B11B.LOCAL = {
        kdc = 10.0.142.2:88
        admin_server = 10.0.142.2:464
        default_domain = B11B.LOCAL
    }

[domain_realm]
    .b11b.local = B11B.LOCAL
    b11b.local = B11B.LOCAL
```

Im Anschluss kann über den Befehl `kinit <User>` ein Ticket angefordert werden. Überprüfung erfolgt mit Hilfe von `klist`!

PAM

Allgemeines

PAM ist die Abkürzung für Pluggable Authentication Modules und stellt grundlegend eine Sammlung von Modulen zur Verfügung, womit Administratoren die Identifizierung und Authentifizierung der Benutzer bei der Systemanmeldung konfigurieren können. Mit Hilfe von PAM ist man in der Lage die Anmeldung nach unterschiedlichen Methoden wie z. B. Passwort, Token oder gar biometrische Merkmale durchzuführen. Weiterhin können auch verschiedene Wege der Anmeldung z. B. Login, FTP, SSH etc. konfiguriert werden. (vgl. [PAM1])

Mit PAM wird versucht, dass die Authentifizierung klar von der eigentlichen Applikation getrennt wird. Es gibt die unterschiedlichsten Programme und natürlich wollen alle wissen, ob ein User wirklich auch derjenige ist, für den er sich ausgibt. Wie sich ein User gegenüber einer Anwendung authentifiziert, ist dabei natürlich auch unterschiedlich. Vielleicht verwendet er einen Benutzernamen und ein Kennwort, welches entweder lokal oder remote mit Hilfe von LDAP und Kerberos gespeichert wird. Vielleicht nutzt er allerdings auch seinen Fingerabdruck oder ein entsprechendes Zertifikat. Es wäre äußerst schwierig, wenn ein Programm jede dieser Authentifizierungsmöglichkeiten beherrschen müsste. Genau an dieser Stelle greift PAM ein. PAM bietet für die unterschiedlichsten Module Authentifizierungsmöglichkeiten. (vgl. [PAM1])

Bei PAM unterscheidet man generell zwischen Modultypen:

- **auth :**
Das Modul `auth` ist verantwortlich für die Authentifizierung von Benutzern anhand der Überprüfung von z. B. der Kombination aus Benutzername und Passwort. In Unix wird hierfür die `pam_unix.so` genutzt, welche Zugriff auf `/etc/passwd` und `/etc/shadow` hat. Für eine Authentifizierung mit Kerberos wird `pam_krb5.so` genutzt. (vgl. [PAM1])
- **account :**
Mit Hilfe des `account` Moduls werden Benutzerkonten auf z. B. Gültigkeit, Zulassung etc. geprüft. (vgl. [PAM1])
- **password :**
Das Modul `password` ist für die Verwaltung und Aktualisierung von Passwörtern zuständig. (vgl. [PAM1])

- `session` :

Wie der Name vermuten lässt, verwaltet das Modul Sitzungsinformationen und -konfigurationen, wie das Erstellen und auch das Aufräumen von Sessions zu einem speziellen Benutzer. (vgl. [PAM1])

Gängige PAM Module (Auszug)

Es existieren eine Vielzahl unterschiedlichster PAM Module, welche natürlich alle einen anderen Verwendungszweck und eine andere Bedeutung haben. Im Folgenden werden einige wichtige Module kurz näher vorgestellt:

- `pam_unix.so` :

Mit diesem Modul können Benutzer- und Passwortauthentifizierungen anhand der beiden Dateien `/etc/shadow` und `/etc/passwd` durchgeführt werden. (vgl. [PAM2])

- `pam_krb5.so` :

Mit Hilfe dieses Moduls wird die Authentifizierung durch Kerberos 5 unterstützt. (vgl. [PAM2])

- `pam_mkhomedir.so` :

Sobald sich ein Benutzer das erste Mal anmeldet, wird durch dieses Modul ein entsprechendes Home-Verzeichnis erstellt. (vgl. [PAM2])

- `pam_tally2.so` :

Das Modul zählt fehlgeschlagene Authentifizierungsversuche und sperrt nach einer gewissen Anzahl den Benutzer. (vgl. [PAM2])

- `pam_env.so` :

Dieses Modul ist in der Lage die unterschiedlichsten Umgebungsvariablen zu setzen und auch zu entfernen. (vgl. [PAM2])

- `pam_deny.so` :

Das Modul `pam_deny.so` bewirkt, dass jede Interaktion, bei der es verwendet wird, fehlschlägt. Es gibt immer ein Fehler zurück, unabhängig von den Umständen. (vgl. [PAM2])

- `pam_permit.so` :

Im Gegensatz zu `pam_deny.so`, bewirkt dieses Modul immer einen Erfolg. (vgl. [PAM2])

Der Aufbau der PAM Konfiguration

Die Konfigurationsdateien für PAM befinden sich unter `/etc/pam.d`. Für die unterschiedlichsten Dienste existieren dabei auch die unterschiedlichsten Konfigurationsdateien. Dateien, die mit `common` beginnen, sind dabei Konfigurationsdateien, die in andere Konfigurationen einfach inkludiert werden. Betrachtet man zum Beispiel die `common-auth` Konfiguration, so sollte diese ungefähr folgendermaßen aussehen:

```
auth    [success=2 default=ignore]    pam_krb5.so
auth    [success=1 default=ignore]    pam_unix.so nullok_secure try_first_pass
auth    requisite                     pam_deny.so
auth    required                      pam_permit.so
```

TERMINAL

Die Datei weist dabei nachfolgenden Aufbau auf:

1. Spalte = Typ des Moduls

Hat man hintereinander mehrere gleiche Typen, so werden diese bei fehlgeschlagenen Authentifizierungen auch nacheinander ausgeführt. Im Beispiel bezieht sich der Eintrag auf den Typen `auth`.

2. Spalte = Kontrolle

Mit Hilfe der Kontrolle wird in der Konfiguration festgelegt, was passieren soll, wenn eine Authentifizierung fehlschlägt. Es gibt auch hier mehrere mögliche Werte: (vgl. [PAM2])

- `requisite` :
Sollte ein Authentifizierungsversuch mit einem Modul fehlschlagen, dann werden auch alle weiteren Typen ebenfalls nicht mehr ausgeführt. Der Prozess endet folglich. (vgl. [PAM2])
- `required` :
Ähnlich wie bei `requisite` wird auch hier die Authentifizierung gestoppt, allerdings werden trotzdem noch die folgenden Typen ausgeführt. Im Endergebnis ist aber die Authentifizierung trotzdem fehlgeschlagen. (vgl. [PAM2])
- `sufficient` :
Das Gegenstück zu `requisite`. Ist ein Modul mit diesem Kontrolltypen erfolgreich, wird die ganze Authentifizierung als erfolgreich angesehen. Grundvoraussetzung ist allerdings, dass alle anderen vorher durchgeführten Module, welche mit `required` gekennzeichnet waren, erfolgreich waren. (vgl. [PAM2])
- `optional` :
Der Erfolg oder das Fehlschlagen dieses Moduls hat keinen Einfluss auf die Gesamtentscheidung, außer wenn kein anderes Modul mit einem höheren Kontrollflag definiert ist. (vgl. [PAM2])

3. Spalte = Modulpfad

Hier befindet sich das entsprechende Modul, welches auch ausgeführt werden sollte. (vgl. [PAM2])

4. Spalte = Parameter

Jedes Modul kann spezifische Parameter haben. Das `pam_unix.so` Modul hat zum Beispiel die beiden Parameter `nullok_secure` und `try_first_pass`. Mit `nullok` wird grundlegend erlaubt, dass auch leere Passwörter genutzt werden dürfen. `nullok_secure` erweitert diese Richtlinie allerdings, dass nur dann leere Passwörter für die Authentifizierung verwendet werden können, wenn der Zugriff über eine sichere Konsole stattfindet. Der Parameter `try_first_pass` versucht, dass das Passwort aus einem vorherigen Authentifizierungsmodul verwendet wird. Ein weiterer möglicher Parameter ist zum Beispiel `minimum_uid=1000`. Hierrüber wird das Minimum der ID von Benutzern festgelegt, welche berücksichtigt werden sollte, wenn ein Modul ausgeführt wird. Dieser Parameter wird häufig verwendet, um bestimmte Benutzer oder Benutzergruppen aus der Authentifizierung auszuschließen oder einzuschließen. (vgl. [PAM2])

Weitere besondere Kontrollmechanismen

Im aufgeführten Beispiel steht zum Beispiel `[success=2 default=ignore]`. Sollte die Authentifizierung erfolgreich gewesen sein, so werden im Beispiel die nächsten beiden Modulprüfungen übersprungen. Das wird über den Parameter `success=2` geregelt. Sollte das Ergebnis weder erfolgreich noch fehlgeschlagen sein, sondern einen anderen Rückgabewert haben, wird das Modul ignoriert und das nächste Modul ausgeführt. Dieses Verhalten wird über `ignore=default` gesteuert. (vgl. [PAM2])

Das Problem mit der Uhrzeit

Kerberos basiert sehr stark auf dem Faktor Zeit. Jedes Ticket, das ausgestellt wird, hat eine gewisse Gültigkeitsdauer und auch eine entsprechende Zeit, bis zu der es erneuert werden kann. In diesem Zuge ist es von großer Bedeutung, dass die Clients, welche die entsprechenden Tickets anfordern, die gleiche Zeit haben wie das entsprechende KDC (Key Distribution Center). Kerberos kann nicht ordnungsgemäß funktionieren, wenn Client und Server unterschiedliche Zeiten haben. Um dies zu ermöglichen, erfolgt eine Zeitsynchronisation. (vgl. [KER3])

Der Domain Controller holt sich seine exakte Uhrzeit von einer externen Zeitquelle im Internet. Bei diesen Zeitquellen handelt es sich in der Regel um Network Time Protocol (NTP) Server. Da der Domain Controller jetzt eine präzise und standardisierte Zeit bereitstellt, kann er als Zeitserver innerhalb eines Netzwerks genutzt werden. Alle Geräte im Netzwerk synchronisieren folglich ihre Zeit mithilfe des Domain Controllers. (vgl. [KER3])

In Windows wird für die Zeitsynchronisierung der Windows Time Service (w32time) verwendet. Über den Befehl `w32tm /query /status` kann der Status des Windows Time Services überprüft werden. Man findet hierbei zum Beispiel heraus, über welchen Zeitserver die aktuelle Uhrzeit bezogen wird. (vgl. [KER3])

`rdate` (deprecated)

Mit Hilfe von `rdate` ist man in der Lage, von einem entsprechenden Server die aktuelle Uhrzeit als UTC (= Universal Time Coordinated) abzufragen und diese ebenfalls als entsprechende Systemzeit zu setzen. Generell bzw. ohne Zusatz im Standard verwendet `rdate` das Time bzw. Daytime Protokoll, welches allerdings als veraltet gilt und durch das Network Time Protokoll (NTP) mittlerweile ersetzt wurde. (vgl. [RDA1])

```
rdate -npv <Name_of_NTP_Server>
```

TERMINAL

- `-n` :
Durch `-n` wird auf das Network Time Protokoll, anstelle des Time bzw. Daytime Protokoll verwendet.
- `-p` :
Der Parameter `-p` steht für die Ausgabe der abgerufenen Zeit vom Server.
- `-v` :
Hierrüber wird eine ausführliche Ausgabe des Kommandos erzielt.

Möchte man wirklich die Systemzeit mit Hilfe des NTP Servers anpassen, so gibt es noch den Parameter `-a`. Über diesen Parameter kann die Systemzeit an die Remotezeit schrittweise angepasst werden, ohne sie sofort zu verschieben.

Synchronisation automatisiert durchführen

Um die Synchronisation mit einem NTP Server und dem Befehl `rdate` automatisiert durchführen zu können, muss ein Job eingerichtet werden:

Package für Jobs installieren

```
apt install cron
```

TERMINAL

Job einrichten

```
crontab -e
```

TERMINAL

```
*/5 * * * * /usr/sbin/rdate -na 0.de.pool.ntp.org >> /var/log/rdate.log 2>&1
```

TERMINAL

Mit dem aufgeführten Kommando wird alle 5 Minuten eine Synchronisation mit dem NTP Server `0.de.pool.ntp.org` durchgeführt. Falls es zu einem Fehler kommen sollte, wird die entsprechende Meldung in eine Logdatei geschrieben.

`timedatectl`

Der Befehl `timedatectl` kann verwendet werden um die Systemzeit abzufragen und auch entsprechend zu setzen.

Zeitzone auflisten

```
timedatectl list-timezones
```

TERMINAL

Zeitzone setzen

```
timedatectl set-timezone <Name_of_Zone>
```

TERMINAL

IMPORTANT

Angenommen die Zeit in Deutschland ist 14:00 Uhr. Die UTC Zeit ist 12:00, da die Mitteleuropäische Sommerzeit - 2 Stunden gleich der UTC ist. Hat man sein System in der Zeitzone `America/Cayman` konfiguriert, ist es 08:00 Uhr, da UTC - 4 Stunden. Würde man nun das `rdate` Kommando ausführen, so würde die UTC Zeit vom NTP Server geholt werden und im Anschluss die Zeit auf die konfigurierte Zeitzone angepasst werden! Es ist also wichtig, zu berücksichtigen, dass `rdate` wirklich nur die UTC Zeit abfragt.

ntpdate (deprecated)

Im Vergleich zu `rdate` ist `ntpdate` wesentlich geauuer und umfangreicher, da es anstelle des Time bzw. Daytime Protokolls das Network Time Protokoll standardmäßig benutzt. Dabei synchronisiert sich der Befehl auch nicht nur mit einem einzigen NTP Server, sondern kann sich mit mehreren Servern synchronisieren. `ntpdate` ist somit für eine genauere und zuverlässigere Zeitsynchronisation konzipiert

```
ntpdate -q <Name_of_NTP_Server>
```

TERMINAL

Mit dem Parameter `-q` kann der entsprechende NTP Server abgefragt werden. Allerdings wird hierbei noch nicht die Uhrzeit gesetzt.

Synchronisation automatisiert durchführen

Um die Synchronisation mit einem NTP Server und dem Befehl `ntpdate` automatisiert durchführen zu können, muss ein Job eingerichtet werden:

Package für Jobs installieren

```
apt install cron
```

TERMINAL

Job einrichten

```
crontab -e
```

TERMINAL

```
* /5 * * * * /usr/sbin/ntpdate de.pool.ntp.org >> /var/log/ntpdate.log 2>&1
```

TERMINAL

Mit dem aufgeführten Kommando wird alle 5 Minuten eine Synchronisation mit dem NTP Server `de.pool.ntp.org` durchgeführt. Falls es zu einem Fehler kommen sollte, wird die entsprechende Meldung in eine Logdatei geschrieben.

ntpd

Bei `ntpd` handelt es sich um einen Daemon, welcher die Systemzeit von einem NTP Server ständig synchronisiert. Im Gegensatz zu `ntpdate` läuft somit die Synchronisation immer, wodurch die Systemzeit immer aktuell ist und dadurch auch etwaige Verzögerungen oder gar Schwankungen berücksichtigt werden.

Daemon einrichten

```
apt install ntp
```

TERMINAL

Sofern `ntp` installiert wurde, kann der aktuelle Status über nachfolgenden Befehl eingesehen werden:

```
systemctl status ntp
```

TERMINAL

NTP konfigurieren

Nach der erfolgreichen Installation von NTP, muss dieser Daemon noch konfiguriert werden. Innerhalb des Verzeichnisses `/etc` existiert dabei eine Datei mit dem Namen `ntp.conf` bzw. in Debian 13 auch unter `/etc/ntpsec/ntp.conf`. Das ist die zentrale Konfigurationsdatei, in welcher auch mehrerer unterschiedliche NTP Server für die Synchronisation spezifiziert werden können. Um einen neuen NTP Server hinzuzufügen, muss nachfolgender Eintrag in der Datei gemacht werden: (vgl. [NTP1])

```
server <Name_of_NTP_Server> iburst
```

TERMINAL

Das Schlüsselwort `iburst` erlaubt eine schneller Synchronisation beim Start des NTP Services.

Eine vollständige Konfigurationsdatei könnte zum Beispiel wie folgt aussehen:

```
# /etc/ntpsec/ntp.conf

# Poolserver, die nacheinandere abgefragt werden
server 0.de.pool.ntp.org iburst
server 1.de.pool.ntp.org iburst
server 2.de.pool.ntp.org iburst
server 3.de.pool.ntp.org iburst

# Optional: Eigene bzw. einzige Server
# server time.google.com iburst

# Driftfile (Standard): Zur Erfassung der entsprechenden Zeitabweichungen
driftfile /var/lib/ntpsec/ntp.drift

# Logging
logfile /var/log/ntpsec/ntp.log

# Einstellungen für den eigenen Rechner
restrict default kod nomodify nopeer noquery notrap
restrict 127.0.0.1
restrict ::1
```

TERMINAL

Erläuterung zur Konfiguration

- **server :**
Hier werden die NTP Server spezifiziert, mit denen sich der Daemon synchronisieren soll.
- **driftfile :**
Die Datei, in der die Drift (Abweichung der Systemuhr) gespeichert wird.
- **logfile :**
Die Datei, in der die Logs des NTP Daemons gespeichert werden.
- **restrict :**
Hierüber können Zugriffsrechte für Clients definiert werden:
 - **kod :**
"kiss-of-death" Paket senden, wenn ein Client zu viele Anfragen sendet.
 - **nomodify :**
Verhindert, dass Clients die Zeit des Servers ändern.
 - **nopeer :**
Verhindert, dass Clients eine Peer-Beziehung mit dem Server aufbauen.
 - **noquery :**
Verhindert, dass Clients Statusabfragen an den Server senden (Scans, Angriffe etc.).

- notrap :

Verhindert, dass Clients Trap-Pakete an den Server senden. Betrifft das Remote-Monitoring.

NTP Service überprüfen

Sofern eine Änderung innerhalb der Konfiguration gemacht wurde, muss der NTP Service neu gestartet werden:

```
systemctl restart ntp
```

TERMINAL

Um im Anschluss zu verifizieren, ob der Service richtig funktioniert, gibt es mehrere unterschiedliche Befehle:

```
ntpq -p
```

```
ntpq -p
```

TERMINAL

Hierrüber wird das NTP Abfrageprogramm gestartet, wodurch der Abfragestatus der eingetragenen NTP Server geprüft werden kann. Als Ausgabe erhält man zum Beispiel:

```
remote          refid  st  t   when   poll   reach  delay  offset  jitter
=====
*time.nist.gov   .NIST.  1   u   64     64    377    24.727  0.123  1.234
+time.google.com .GOOG.  1   u   59     64    377    24.394  0.456  1.567
```

TERMINAL

NOTE

* steht für den aktuell genutzten Server für die Zeitsynchronisation. + hingegen kennzeichnet die Server, die als Alternativen zur Verfügung stehen.

- remote :
Der Name des NTP-Servers.
- refid :
Der Referenz-ID des Servers.
- st :
Die Stratum-Stufe des Servers. Hierrüber wird die Entfernung zum NTP Server spezifiziert.
- when :
Die Zeit seit der letzten Synchronisation (in Sekunden).
- poll :
Das Intervall zwischen den Abfragen (in Sekunden). * reach :
Eine Bitmaske, die zeigt, wie oft der Server erreicht wurde.
- delay :
Die Netzwerkverzögerung (in Millisekunden).
- offset :
Die Zeitdifferenz zwischen dem lokalen Rechner und dem Server (in Millisekunden).
- jitter :
Die Zeitabweichung (in Millisekunden).

```
ntptime
```

Der Befehl `ntptime` kann die Konfiguration testen indem die Systemaufrufe `ntp_gettime()` und `ntp_adjtime()` aufgerufen werden. Falls das erfolgreich sein sollte, bekommt man die entsprechenden Zeitinformatoren zurück:

```
ntptime
```

TERMINAL

```
ntp_gettime() returns code 0 (OK)
  time ea4623c4.de414e80 2024-07-20T11:53:40.868Z, (.868184212),
  maximum error 88016 us, estimated error 16 us, TAI offset 37
ntp_adjtime() returns code 0 (OK)
  modes 0x0 (),
  offset 0.000 us, frequency -402.870 ppm, interval 1 s,
  maximum error 88016 us, estimated error 16 us,
  status 0x6001 (PLL,NANO,MODE),
  time constant 0, precision 1.000 us, tolerance 500 ppm,
```

TERMINAL

systemd-timesyncd

`systemd-timesyncd` ist ein NTP Client, der nahtlos in das Systemmanagement von `systemd` integriert ist. Ebenfalls wie `ntpdate` und `ntpd` nutzt der Dienst das Network Time Protokoll um sich mit entsprechenden NTP Servern zu synchronisieren.

Konfiguration

Um den Service `systemd-timesyncd` auf einem System einsetzen zu können, muss dieser zunächst installiert werden:

```
apt install systemd-timesyncd
```

TERMINAL

Die eigentliche Konfiguration des Services kann in der Datei `/etc/systemd/timesyncd.conf` gefunden werden. Wesentliche Einstellungen sind dabei:

```
[Time]
NTP=0.debian.pool.ntp.org 1.debian.pool.ntp.org
FallbackNTP=2.debian.pool.ntp.org 2.debian.pool.ntp.org
```

TERMINAL

- **NTP :**
Die Server, mit denen der Service versucht sich zu synchronisieren.
- **Fallback :**
Sollten die Server unter dem Punkt `NTP` nicht erreichbar sein, so greifen die sekundären NTP Server.

Überprüfung des Services

Wie bereits in Kapitel 2.2.6.1 angesprochen, kann der Befehl `timedatectl` genutzt werden, um zu überprüfen, ob die entsprechende Synchronisation funktioniert:

```
Local time: Sa 2024-07-20 14:30:44 CEST
Universal time: Sa 2024-07-20 12:30:44 UTC
RTC time: Sa 2024-07-20 12:30:44
Time zone: Europe/Berlin (CEST, +0200)
System clock synchronized: yes
NTP service: active
RTC in local TZ: no
```

TERMINAL

Interessant sind dabei die beiden Punkte `System clock synchronized` und `NTP service`. Durch die Parameterwerte `yes` und `active` wird angegeben, dass die Synchronisation über den Service funktioniert.

LDAP

LDAP ist die Abkürzung für Lightweight Directory Access Protocol. Das Protokoll selbst dient für den Zugriff und die Verwaltung von verteilten Verzeichnisinformationen. Das können zum Beispiel Benutzer, Gruppen oder andere Ressourcen wie Dateien und Drucker innerhalb eines Netzwerks sein. Das LDAP Verzeichnis ist in diesem Zusammenhang eine hierarchische Datenbank, die Informationen über Benutzer, Gruppen, Geräte und andere Ressourcen speichert. Häufigster Einsatzbereich von LDAP ist die Ablage von Benutzernamen und entsprechenden Passwörtern. (vgl. [LDA1])

Grundbegriffe

Innerhalb des LDAP Verzeichnisses besteht ein Eintrag immer aus einer Reihe von unterschiedlichen Attributen:

```
dn: uid=jdoe,ou=users,dc=example,dc=com
cn: John Doe
sn: Doe
uid: jdoe
mail: jdoe@example.com
```

TERMINAL

dn

dn ist die Abkürzung für Distinguished Name und ist der eindeutige Name für einen Eintrag im LDAP Verzeichnis. Der Name selbst besteht aus einer Reihe unterschiedlichster Attribute. Im Beispiel ist der dn folgendermaßen:

```
dn: uid=jdoe,ou=users,dc=example,dc=com
```

TERMINAL

- **uid :**
uid ist die Abkürzung für Universally Unique Identifier und steht für eine entsprechende eindeutige Kennzeichnung eines Objekts. Im Beispiel wird hierfür der Benutzername des Users verwendet.
- **ou :**
ou steht für Organisational Unit. Organisational Units sind Organisationseinheiten, die mehrere andere Objekte wie Benutzer, Gruppen oder andere Ressourcen zusammenfassen. Man spricht auch häufig von Container Objekten.
- **dc :**
Die Domain Component wird anhand einer vollständigen Domäne gebildet. Lautet die Domäne zum Beispiel `example.com` so ergeben sich gesamt zwei dc Einträge:
 - `dc=example`
 - `dc=com`

RDN

Der Relative Distinguished Name ist ein Teil des herkömmlichen Distinguished Names, der allerdings einen Eintrag innerhalb des LDAP Verzeichnisses wirklich eindeutig identifiziert. Im aufgeführten Beispiel ist das die uid des Benutzers.

Weitere Attribute

- **cn**
Der Common Name ist der vollständige Name eines Benutzers, welcher sich aus Vor- und Nachname zusammensetzt. Im Beispiel John Doe .
- **sn**
Der Surname, also der Nachname, eines Benutzers.

- **uid**
Erneute Auflistung des Universally Unique Identifiers. Im Beispiel `jdoe`.
- **mail**
Die E-Mail Adresse des Benutzers.

Attribute, Objektklassen und Schemas

Wie bereits angesprochen, besteht jeder Eintrag innerhalb des LDAP Verzeichnisses aus einer Reihe von unterschiedlichen Attributen. Welche Attribute ein Eintrag dabei hat und welche davon Pflichtattribute und welche optionale Attribute sind, wird durch die Objektklassen festgelegt. Objektklassen selbst stellen somit mehr oder weniger einen Rahmen für Einträge im LDAP Verzeichnis dar. (vgl. [LDA2])

Damit ein LDAP Verzeichnis auch an eine entsprechende Unternehmensstruktur angepasst werden kann, existieren Schemas, die auch um eigene Objektklassen und Attributstypen erweitert werden können. Die Schemas innerhalb von LDAP haben dabei den Vorteil, dass alle Einträge im Verzeichnis eine konsistente und vorhersehbare Struktur haben. Auch die Suche nach Einträgen und die Verwaltung der Verzeichnisse wird vereinfacht, da bekannt ist, welche Attribute vorhanden sein sollten und welche Werte sie annehmen können. (vgl. [LDA2])

Zugriff auf das LDAP Verzeichnis

Installation der entsprechenden Toolings

```
apt install ldap-utils
```

TERMINAL

Suche innerhalb des LDAP Verzeichnisses

```
ldapsearch -x -H ldap:///<IP_of_LDAP_Server> -b <Search> -D <User_for_Search> -w <Password_for_User>
```

TERMINAL

- **ldapsearch :**
Das eigentliche Kommando für die Suchanfrage innerhalb des LDAP Verzeichnisses.
- **-x :**
Verwendet die einfache Authentifizierung (simple authentication) anstelle der SASL-Authentifizierung (Simple Authentication and Security Layer).
- **-H ldap:///<IP_of_LDAP_Server> :**
Angabe des Hosts (= IP des LDAP Servers) mit dem eine Verbindung hergestellt werden sollte.
- **-b :**
Der BasisDN, also der Distinguished Name, unter dem die eigentliche Suche begonnen werden sollte.
- **-D :**
Der AnmeldeDN, also der Distinguished Name, mit dem die Anmeldung auf dem LDAP Server durchgeführt werden sollte.
- **-w :**
Das Passwort für den spezifizierten Benutzer innerhalb des AnmeldeDN.

Als Ergebnis erhält man alle gefundenen Einträge innerhalb des LDAP Verzeichnisses, die unter dem entsprechenden BasisDN gefunden werden. Die Ergebnismenge schließt dabei auch alle entsprechenden Attribute und die dazugehörigen Werte mit ein.

Abwandlungen für die Suche

```
ldapsearch -x -LLL -H ldap:///<IP_of_LDAP_Server> -b <Search> -D <User_for_Search> -w <Password_for_User>
```

TERMINAL

- -LLL :

Dieser Parameter beeinflusst sehr stark die Ausgabe der `ldapsearch`. Ein einzelnes `L` limitiert die Ausgabe in auf das LDIFv1 Format. Ein zweites `L` entfernt etwaige Kommentare und das dritte `L` entfernt Informationen wie zum Beispiel die LDIF Version.

```
ldapsearch -x -LLL -H ldap://<IP_of_LDAP_Server> -b <Search> -D <User_for_Search> -w <Password_for_User> "(Pattern_for_Search)" cn
```

- "(Pattern_for_Search)"

Hier kann ein entsprechendes Suchmuster erfasst werden. Das Suchmuster besteht dabei immer aus einem Attribut und dem entsprechenden Wert. Ein Beispiel könnte zum Beispiel sein `"(cn=Mori*)"`. Hierüber wird innerhalb des LDAP Verzeichnisses ein Eintrag mit einem speziellen Common Name gesucht. Der Name beginnt dabei mit `Mori` und darauf folgen beliebig viele andere Zeichen.

- cn :

Hierrüber wird die eigentliche Ausgabe beeinflusst. Es wird auf die erfolgreiche Suche lediglich der Common Name ausgegeben.

Beispiele für die Suche

```
ldapsearch -x -LLL -H ldap://b11b.dvs-plattling.de -b OU=BFS2024IK,OU=Klassen,OU=Benutzer,DC=b11b,DC=local -D CN=sheitzer,OU=Lehrer,OU=Benutzer,DC=b11b,DC=local -w <Password_for_User>
```

Die Einrichtung eines eigenen LDAP Servers

Installation unter Linux

Um einen eigenen LDAP Server betreiben zu können, müssen zunächst entsprechende Pakete installiert werden:

```
apt update
apt install slapd ldap-utils
```

Während der Installation von `slapd` werden einige Informationen abgefragt:

- Administrator password :

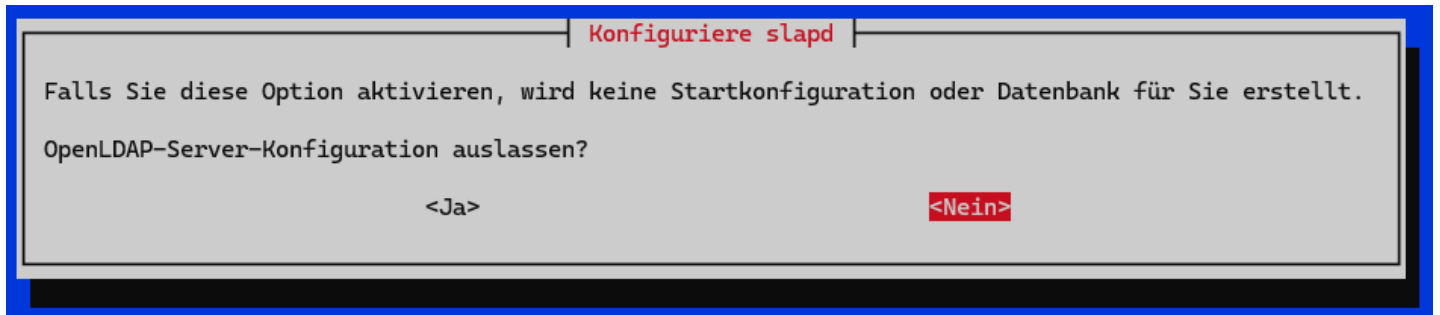
Das Passwort für den Administrator, der für das LDAP Verzeichnis bzw. den Server verantwortlich ist. Hierfür kann ein freier Wert vergeben werden wie z. B. `egon34`. Im Anschluss muss das Passwort bestätigt werden.

Nach der erfolgreichen Installation der Pakete kann nachfolgender Befehl ausgeführt werden:

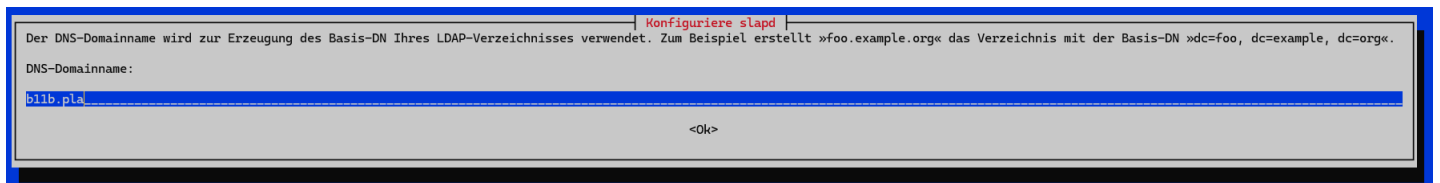
```
dpkg-reconfigure slapd
```

TERMINAL

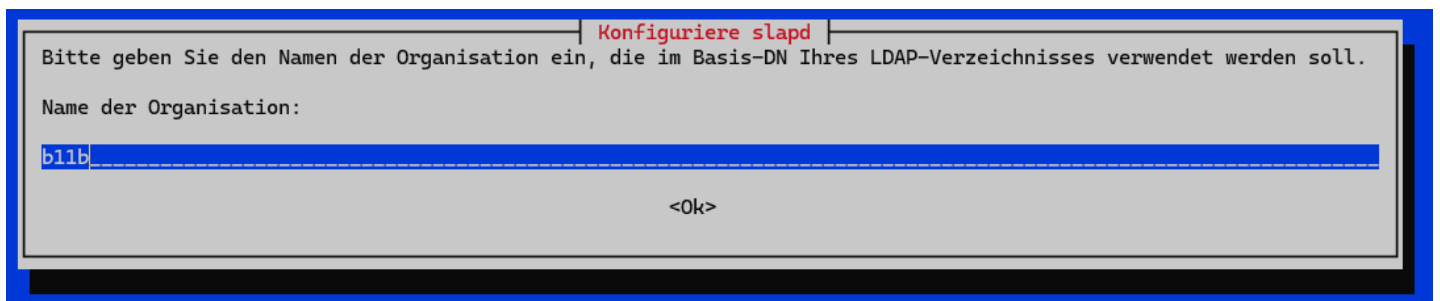
Mit Hilfe dieses Befehls wird die Konfiguration des Stand-alone LDAP Daemons eingeleitet. `slapd` fragt zu Beginn, ob die OpenLDAP Konfiguration verworfen werden soll, oder nicht. Hier mit `Nein` bzw. `No` antworten:



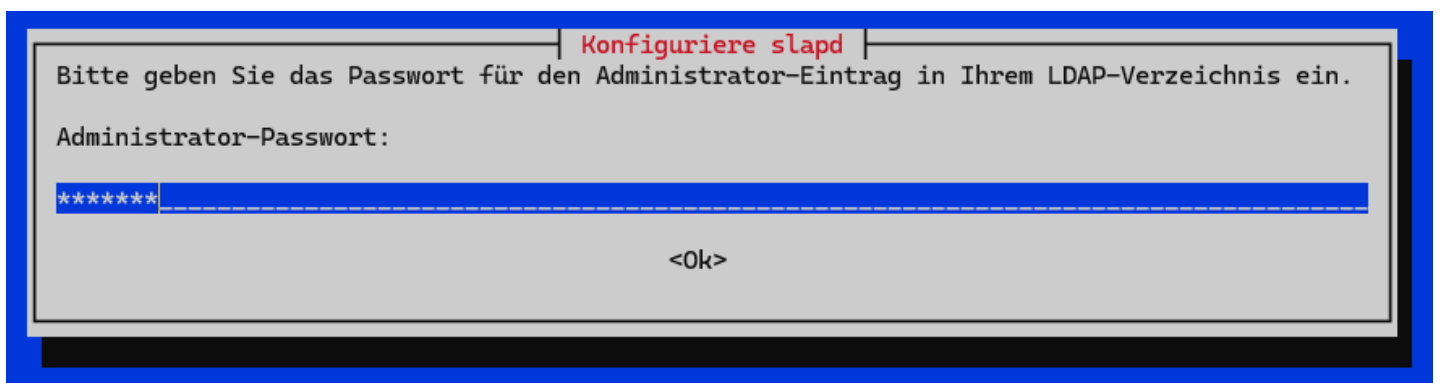
Im Anschluss wird der DNS Domain Name verlangt. Hier sollte man die einzelnen DN Komponenten wieder zusammensetzen. Wird zum Beispiel als DN `dc=example,dc=net` dann ist der entsprechende DNS Domain Name `example.net`:



Nachdem der DNS Domain Name gepflegt wurde, muss der Name der Organisationseinheit gepflegt werden:

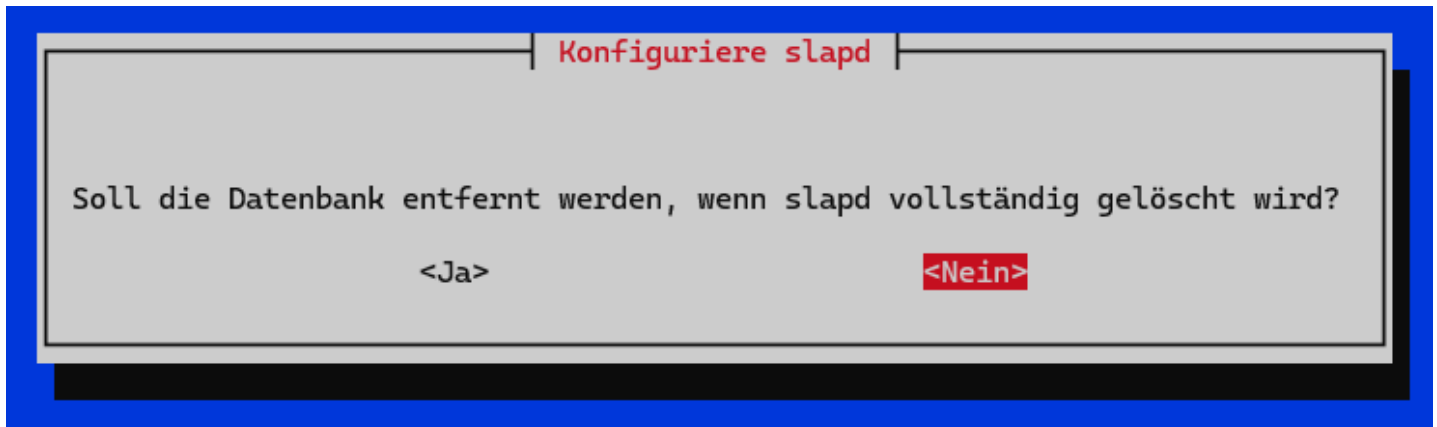


`slapd` verlangt während der Konfiguration erneut nach einem Passwort für den Administrator des LDAP-Verzeichnisses. Auch dieses muss im Anschluss erneut bestätigt werden:





Zum Schluss wird man gefragt, ob die Datenbank verworfen werden sollte, wenn `slapd` entfernt wird. Hier ebenfalls mit Nein bzw. No bestätigen:



Im Hintergrund existieren eine relevante Datei, die im Zuge der Einrichtung angepasst wird bzw. angepasst werden muss:

`/etc/ldap/ldap.conf`

Die Datei `/etc/ldap/ldap.conf` ist die zentrale Konfigurationsdatei für die Verwendung und den Betrieb eines LDAP Servers. Mit der Datei wird festgelegt, wie Suchen durchgeführt werden und wie auch die Sicherheit für die Verbindungen gewährleistet wird. Relevante Einstellungen sind:

- **BASE :**
Für die Suchbasis im LDAP Verzeichnis muss ein entsprechender Distinguished Name angegeben werden. Viele Unternehmen verwenden hierfür die Komponenten ihrer Domain. Sollte die Domain zum Beispiel `example.net` sein, so wird der DN aufgesplittet in `dc=example,dc=net`
- **URI :**
Die URI unter der der LDAP Server erreicht werden kann. Das Format ist dabei immer `ldap://<hostname_or_IP_address:port>/`

Die Konfiguration sollte im Anschluss nachfolgenden, beispielhaften Aufbau haben:

```
#
# LDAP Defaults
#

# See ldap.conf(5) for details
# This file should be world readable but not world writable.

BASE    dc=b11b,dc=pla
URI      ldap://127.0.0.1

#SIZELIMIT      12
#TIMELIMIT      15
#DEREF          never

# TLS certificates (needed for GnuTLS)
TLS_CACERT      /etc/ssl/certs/ca-certificates.crt
```

Nach all diesen Einstellungen kann im Anschluss die LDAP-Verbindung getestet werden:

Aufruf am LDAP Server:

```
ldapsearch -x -H ldap://127.0.0.1 -b dc=b11b,dc=pla
```

Aufruf über anderen Rechner

```
ldapsearch -x -H ldap://<IP> -b dc=b11b,dc=pla
```

Die Befüllung des LDAP

Um LDAP zu befüllen, werden sogenannte `ldif` Dateien benötigt. Die LDAP Data Interchange Format Dateien sind Textdateien, womit Einträge hinzugefügt, geändert oder gelöscht werden können. In Bezug auf das Format weisen sie pro Zeile immer einen Attributnamen gefolgt von einem Doppelpunkt und dem Attributwert auf. Einträge sind durch Leerzeilen voneinander getrennt.

Hinzufügen neuer Organisationseinheiten

Um innerhalb des LDAP Verzeichnisses einen neuen Container bzw. eine neue Organisationseinheit hinzufügen zu können, wird nachfolgende Syntax benötigt:

```
dn: ou=<Name_of_Container>,dc=<Name_of_Domain_Component>
ou: <Name_of_Organisational_Unit>
objectClass: top
objectClass: organizationalUnit
```

- **dn :**
Der Distinguished Name ist erneut der vollständige Name unter welchem der entsprechende Eintrag im LDAP Verzeichnis angesprochen werden kann.
- **ou :**
Die Organisational Unit ist der Name der Organisationseinheit. Hierunter können folglich weitere Objekte wie Benutzer oder auch andere Organisationseinheiten gruppiert werden.
- **objectClass :**
Die Objektklasse gibt die entsprechenden Attribut an, welche für den Eintrag verwendet werden können. `top` steht dabei für die oberste Hierarchieebene und `organizationalUnit` spezifiziert, dass es sich bei dem Eintrag um eine Organisationseinheit handelt.

Hinzufügen neuer POSIX-Gruppen

POSIX ist die Abkürzung für Portable Operating System Interface und eine Sammlung von diversen Standards, damit die Kompatibilität und Interoperabilität von Betriebssystem sichergestellt wird. Dadurch verhalten sich mehr oder weniger alle Linux System auch identisch. Die Gruppe selber ist dabei nichts anderes als ein Container, die unterschiedliche Benutzer halten kann:

```
dn: cn=<Common_Name_of_Group>,ou=<Name_of_Organizational_Unit>,dc=<Name_of_Domain_Component>
objectClass: posixGroup
objectClass: top
gidNumber: <ID_of_Group>
memberUid: <ID_of_User>
cn: <Common_Name_of_Group>
```

TERMINAL

Neue Attribute im Zusammenhang mit POSIX-Gruppen sind:

- **gidNumber :**
Die eindeutige ID der jeweiligen Gruppen. Wird von POSIX-Gruppen benötigt.
- **memberUid :**
Die eindeutige ID des Users, welcher Mitglied innerhalb der Gruppe werden sollte.

Erstellung neuer Benutzer innerhalb des LDAP Verzeichnisses

Auch für die Erstellung eines neuen Benutzers kann eine `ldif` Datei verwendet werden. Ebenso für Benutzer gibt es einige wesentliche Eigenschaften:

```
dn: uid=<UID_of_new_User>,ou=<Name_of_Organizational_Unit>,dc=<Name_of_Domain_Component>
sn: <Surname_of_new_User>
givenName: <Firstname_of_new_User>
uid: <UID_of_new_User>
cn: <Common_name_of_new_User>
gidNumber: <ID_of_Group_for_User>
uidNumber: <Number_of_UID>
homeDirectory: <Home_Directory_of_User>
objectClass: top
objectClass: posixAccount
objectClass: shadowAccount
objectClass: inetOrgPerson
```

TERMINAL

- **gidNumber :**
Hierbei handelt es sich um die Nummer der Gruppe, zu welcher der neue User hinzugefügt werden sollte. Relevant für POSIX.
- **uidNumber :**
Ähnlich wie die `gidNumber` ist das die eindeutige Nummer des neuen Benutzers, welche er erhalten sollte. Ebenfalls relevant für POSIX.
- **homeDirectory :**
Hierrüber wird der Pfad zum Standardverzeichnis des neuen Benutzers spezifiziert.
- **objectClass :**
Wie bereits in den Punkten für Organisationseinheiten und POSIX-Gruppen, gibt es auch hier die unterschiedlichen Klassen, die vergeben werden können:
 - **top :**
`top` steht wieder für die höchste Organisationsebene.

- `posixAccount` :
Wie bereits beschrieben ist POSIX eine Sammlung von Standards, damit Kompatibilität und Interoperabilität zwischen Betriebssystem gewährleistet wird. Mit dieser Objektklasse wird gekennzeichnet, dass dieser Account nach dem POSIX Standards funktioniert.
- `shadowAccount` :
Diese Objektklasse erlaubt es, dass Passwort- und Sicherheitsinformationen für ein Benutzerkonto gespeichert werden können.
- `inetOrgPerson` :
Mit Hilfe dieser Objektklasse wird eine breite Palette an Attributen angeboten, damit Benutzerkonten wesentliche genauer spezifiziert werden können.

Erstellung der Dateien und tatsächliches Ausführen

Natürlich können innerhalb einer `ldif` Datei auch mehrere Einträge hinzugefügt werden. Wichtig ist allerdings, dass man eine entsprechende Abhängigkeit beachtet! Die Dateien werden von oben nach unten verarbeitet! Um letztendlich wirklich die entsprechenden Einträge in das LDAP Verzeichnis hinzufügen zu können, ist nachfolgender Befehl notwendig:

```
ldapadd -x -D <User_who_adds> -w <Password_for_User_who_adds> -f <Name_of_ldif_File>
```

TERMINAL

Beispielhafte Datei `ldap.ldif`

```
# Container fuer user
dn: ou=People,dc=b11b,dc=pla
ou: People
objectClass: top
objectClass: organizationalUnit

#Container fuer Gruppen
dn: ou=Group,dc=b11b,dc=pla
ou: Group
objectClass: top
objectClass: organizationalUnit

#Erster User
dn: uid=sheitzer,ou=People,dc=b11b,dc=pla
sn: Heitzer
givenName: Stefan
uid: sheitzer
cn: Stef Heitzer
gidNumber: 10000
uidNumber: 2001
homeDirectory: /home/sheitzer
objectClass: top
objectClass: posixAccount
objectClass: shadowAccount
objectClass: inetOrgPerson

#Zweiter User
dn: uid=mmustermann,ou=People,dc=b11b,dc=pla
sn: Mustermann
givenName: Max
uid: mmustermann
cn: Max Mustermann
gidNumber: 10001
uidNumber: 2002
homeDirectory: /home/mmustermann
objectClass: top
objectClass: posixAccount
objectClass: shadowAccount
objectClass: inetOrgPerson

#Gruppe 1
dn: cn=grp1,ou=Group,dc=b11b,dc=pla
objectClass: posixGroup
objectClass: top
gidNumber: 10000
memberUid: sheitzer
cn: grp1

#Gruppe 2
dn: cn=grp2,ou=Group,dc=b11b,dc=pla
objectClass: posixGroup
objectClass: top
gidNumber: 10001
memberUid: mmustermann
cn: grp2
```

Die Datei kann im Anschluss über nachfolgenden Befehl hinzugefügt werden:

```
ldapadd -x -D cn=admin,dc=b11b,dc=pla -w egon34 -f ldap.ldif
```

Anbindung an den LDAP Server

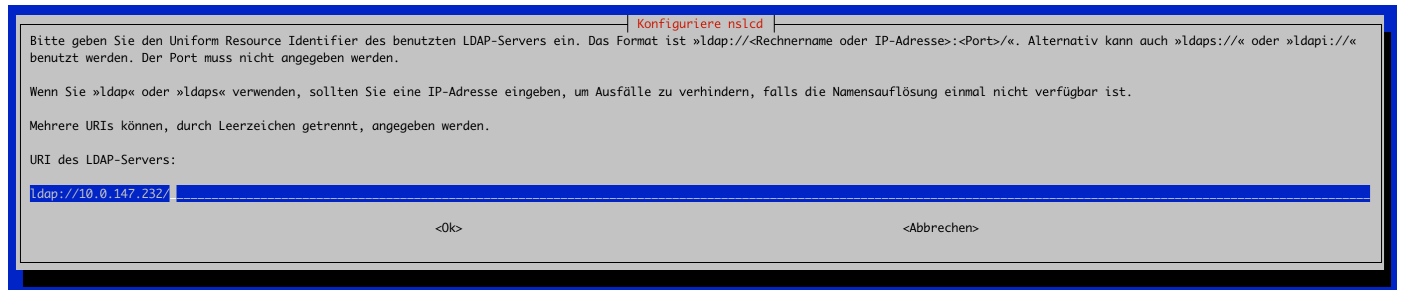
Um einen Client mit einem LDAP Server verbinden zu können, ist es notwendig, dass auf dem Client auch entsprechende Pakete installiert werden:


```
apt update
apt install libpam-ldapd nslcd
```

Während der Installation von `nslcd` müssen einige Informationen gepflegt werden:

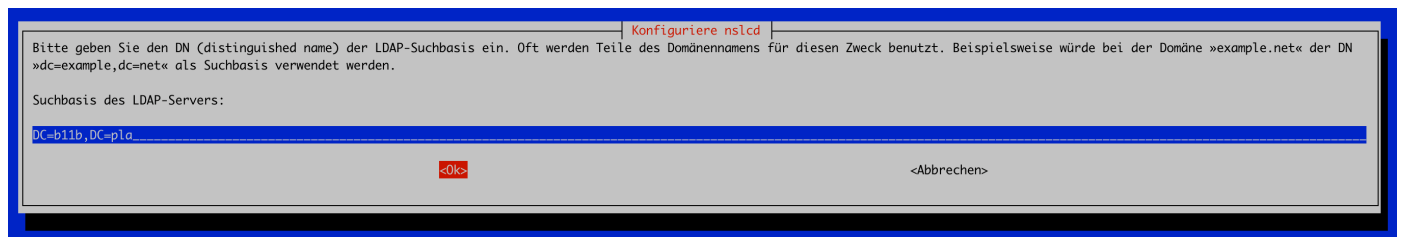
- LDAP Server URI :

Die URI unter der der LDAP Server erreicht werden kann. Das Format ist dabei immer `ldap://<hostname_or_IP_address:port>/`



- DN der LDAP Suchbasis :

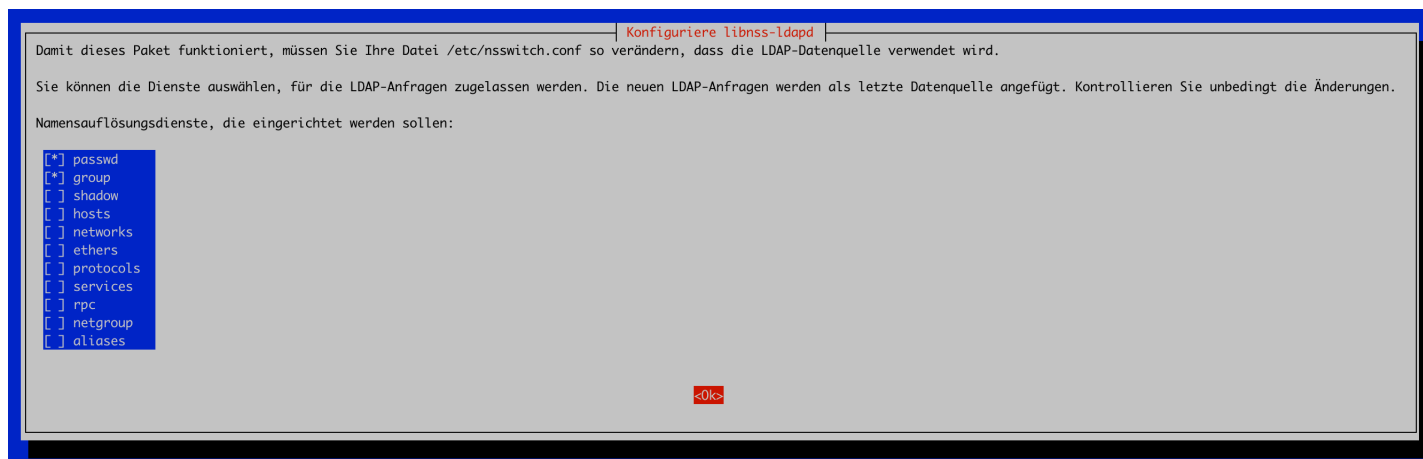
Für die Suchbasis im LDAP Verzeichnis muss ein entsprechender Distinguished Name angegeben werden. Viele Unternehmen verwenden hierfür die Komponenten ihrer Domain. Sollte die Domain zum Beispiel `example.net` sein, so wird der DN aufgesplittet in `dc=example,dc=net`. Wichtig ist, dass hier der gleiche DN wie beim Server verwendet wird.



Name Service Switch (NSS)

Für die Anbindung eines Clients an einen LDAP Server ist es notwendig, dass der Name Service Switch richtig konfiguriert ist. Beim Name Service Switch handelt es sich um eine spezielle Konfiguration, mit welcher Namen und andere Informationen aufgelöst werden können, die von Systemdiensten und entsprechenden Anwendungen benötigt werden. Die eigentliche Konfiguration befindet sich unter `/etc/nsswitch.conf`. Innerhalb der Konfiguration hat man somit die Möglichkeit festzulegen, welche Dienste (z.B. DNS, LDAP, lokale Dateien, etc.) zur Namensauflösung und für andere Systeminformationen wie Benutzerkonten, Gruppen, Hosts, Netzwerke, etc. verwendet werden sollen. (vgl. [NSS1])

Während der Anbindung eines Clients an den LDAP Server, wird im Zuge der Paketinstallation auch diese Datei angepasst. Es gibt einen eigenen Konfigurationsschritt innerhalb des Pakets `libnss-ldapd`, in welchem Services selektiert werden können, die für die Informationsauflösung LDAP verwenden sollen. Für die Grundkonfiguration reicht es aus, die Services `passwd` und `group` zu verwenden:



Auf einem entsprechenden Client, der an den LDAP Server angebunden werden sollte, wird folglich die Datei `/etc/nsswitch.conf` angepasst. Wurde bei der Konfiguration `passwd` für Benutzer und `group` für Gruppen gewählt, so passt sich der Inhalt der Datei wie folgt an:

```
...
passwd:      files systemd ldap
group:       files systemd ldap
...
```

TERMINAL

Die Einstellung bedeutet, dass zunächst entsprechende Dateien auf dem System für die Abfrage nach Benutzern (`/etc/passwd`) und Gruppen (`/etc/group`) herangezogen werden. Sollten hierbei keine Treffer gefunden werden, wird der `systemd` Dienst mit seiner Benutzer- und Gruppendatenbank als Quelle herangezogen. Sollten auch hier keine Treffer gefunden werden, so wird im Anschluss der entsprechende LDAP Server herangezogen.

`/etc/nslcd.conf`

Bei dieser Datei handelt es sich um die Konfigurationsdatei für den NSLCD-Dienst in Linux. Diese Konfiguration regelt, dass überhaupt Benutzerdaten und etwaige Verzeichnisdienste aus einem LDAP Verzeichnis bzw. von einem LDAP Server abgerufen werden können. Wesentliche Einstellungen sind dabei:

- `uri` :
Die `uri` ist die Adresse mit welcher der LDAP Server erreicht werden kann. Wichtig ist, dass das Format `ldap://<hostname_or_IP_address:port>/` eingehalten wird.
- `base` :
Hierbei handelt es sich um den Basis DN, der für alle Suchanfragen verwendet wird.

PAM

Auch in Bezug auf PAM müssen entsprechende Module angepasst werden. Die Module sind `account`, `auth` und `common-session`.

`/etc/pam.d/common-account`

Die `common-account` ist zuständig für den Benutzerprüfung. Es wird geprüft, ob ein Account überhaupt zugelassen oder gar eingeschränkt ist. Innerhalb des Moduls wird eine Zeile für LDAP benötigt und eine bestehende muss angepasst werden:

```
...
account [success=2 new_authtok_reqd=done default=ignore]    pam_unix.so
account [success=1 default=ignore]    pam_ldap.so
...
```

TERMINAL

Als erstes wird `pam_unix.so` genutzt, damit oben genannte Prüfung durchgeführt werden kann. Sollte das erfolgreich sein, so wird die nächste Zeile übersprungen. Im Falle, dass allerdings die erste Modulprüfung scheitern sollte, so wird die zweite Zeile ausgeführt. Hierfür ist das Modul `pam_ldap.so` verantwortlich, welche die Authentifizierung gegen ein LDAP Verzeichnis durchführt.

```
/etc/pam.d/common-auth
```

Die `common-auth` ist generell für den Authentifizierungsprozess selber verantwortlich. Innerhalb des Moduls muss ebenso eine Zeile für LDAP hinzugefügt werden:

TERMINAL

```
...  
auth    [success=1 default=ignore] pam_ldap.so minimum_uid=1000 use_first_pass  
...
```

```
/etc/pam.d/common-session
```

Die `common-session` ist in Modul, welches verantwortlich für die Verwaltung von Sitzungen ist. Das Modul definiert, was beim Start und beim Ende einer Benutzersitzung durchgeführt werden soll. In Bezug auf LDAP soll dabei automatisiert für einen Benutzer ein entsprechendes Standardverzeichnis erstellt werden:

TERMINAL

```
...  
session optional      pam_ldap.so  
session required      pam_mkhomedir.so skel=/etc/skel/ umask=0022  
...
```

Verbindung testen

Sind alle relevanten Einstellungen durchgeführt worden, kann die Verbindung getestet werden:

Cache löschen und Dienst neustarten

TERMINAL

```
nscd -i passwd  
nscd -i group  
systemctl restart nscd  
systemctl restart nslcd
```

Lokale User und Gruppen abrufen

Bei der Ausführung dieses Befehls sollte man noch keine Benutzer und Gruppen des LDAP Servers sehen:

TERMINAL

```
#Benutzer anzeigen  
cat /etc/passwd  
  
#Gruppen anzeigen  
cat /etc/group
```

Lokale und LDAP Server User und Gruppen abrufen

Hierfür wird der Befehl `getent` genutzt. Dieser Befehl ist in der Lage Informationen aus administrativen Datenbanken auszulesen, die innerhalb der `/etc/nsswitch.conf` spezifiziert sind:

TERMINAL

```
#Benutzer anzeigen  
getent passwd  
  
#Gruppen anzeigen  
getent group
```

Fehlerprotokoll auslesen

```
journalctl | grep -i ldap
```

Integration mit Windows Active Directory

In komplexen Umgebungen werden sowohl Linux als auch Windows Systeme verwendet. Folglich ist es relevant, dass eine funktionierende Authentifizierung und Benutzerverwaltung etabliert wird. Innerhalb größerer Betriebe wird hierfür die Authentifizierung mittels Kerberos und die Benutzerverwaltung via LDAP durchgeführt. In Bezug auf die Authentifizierung gibt es grundlegend keine großartigen Probleme. Wichtig ist nur, dass die `krb5.conf` richtig konfiguriert und aktiv ist. Bei der Benutzerverwaltung sieht es allerdings etwas anders aus. Das Problem ist, dass zwischen Linux und Windows verschiedene Attribute für Benutzer und Gruppen nicht identisch sind.

Typ	Windows	Linux
Login	samAccountName	uid
UserID	ObjectSID	uidNr
Home	homeDirectory	homeDirectory

Um die Integration trotzdem erfolgreich durchführen zu können, gibt es mehrere unterschiedliche Ansätze:

- **Eigener LDAP Server:**
Es kann unter Linux ein eigener LDAP Server eingerichtet werden, welcher die identischen Informationen hält wie auch der entsprechende Windows Domänen Controller. Der Aufwand diesbezüglich ist nur, dass jeder einzelne Linux Client auch in Bezug auf LDAP konfiguriert und zusätzlich der entsprechende Server immer wieder synchronisiert werden muss.
- **Samba und winbind unter Linux installieren:**
Der Name Samba leitet sich vom Server Message Block Netzwerkprotokoll ab und ist ein Softwarepaket, welches darauf ausgelegt ist, dass Windows Funktionen wie zum Beispiel Datei- und Druckdienste unter anderen Betriebssystemen genutzt werden. Wesentlich ist auch, dass ein Rechner mit installiertem Samba Softwarepaket auch die Rolle eines Domänen Controllers annehmen bzw. Mitglied innerhalb einer Windows Domäne werden kann. Winbind ergänzt Samba, indem es Benutzer- und Gruppeninformationen von einem Windows-Domänencontroller auf Unix-/Linux-Systeme überträgt und eine zentrale Authentifizierung ermöglicht. (vgl. [SAM1])
- **SFU (Services for Unix):**
Bei den Services for Unix handelt es sich um eine spezielle Erweiterung für das Active Directory. Es werden entsprechend Schemata erweitert, damit die Unix Attribute, welche verschieden sind zu Windows, auch abgebildet werden. Die Einrichtung ist allerdings eher aufwendig.

Vorbereitende Schritte

Um die komplette Integration durchführen zu können, werden einige Pakete benötigt:

```
apt update  
apt install krb5-user libpam-krb5 winbind samba smbclient libnss-winbind libpam-winbind
```

Kerberos einrichten

Nach der erfolgreichen Installation der Pakete kann man mit der Konfiguration von Kerberos starten. Hierzu die Datei `/etc/krb5.conf` mit folgendem Inhalt ersetzen:

```
[libdefaults]
    default_realm = B11B.LOCAL
    forwardable = true
    proxiable = true
    rdns = false
    ticket_lifetime = 24000
    clocks skew = 600
    dns_lookup_kdc = false

[realms]
    B11B.LOCAL = {
        kdc = 10.0.142.2:88
        admin_server = 10.0.142.2:464
        default_domain = B11B.LOCAL
    }

[domain_realm]
    .b11b.local = B11B.LOCAL
    b11b.local = B11B.LOCAL
```

Sofern die Konfiguration eingetragen worden ist, kann die Authentifizierung getestet werden:

```
kinit <User>@B11B.LOCAL
```

Man wird im Anschluss aufgefordert für den entsprechenden User das Passwort einzutragen. Sollte die Authentifizierung erfolgreich gewesen sein, so kann man mit dem Befehl `klist` überprüfen, ob ein entsprechendes Kerberos Ticket ausgestellt wurde!

Domäne beitreten

Anpassen der Samba Konfiguration

Um einer Domäne beitreten zu können muss zunächst die Samba Konfiguration angepasst werden. Diese befindet sich innerhalb der Datei `/etc/samba/smb.conf` :

```
[global]
    security = ads
    realm = b11b.local
    password server = 10.0.142.2
    workgroup = B11B
    kerberos method = secrets and keytab
    winbind use default domain = yes
    winbind refresh tickets = yes
    winbind enum users = yes
    winbind enum groups = yes
    template shell = /bin/bash
    idmap config * : range = 10000 - 19999
    idmap config B11B : backend = rid
    idmap config B11B : range = 100000 - 199999
    inherit acls = Yes
```

- `security = ads` :

Mit dieser Einstellung arbeitet Samba im Active Directory Modus. Der Rechner wird somit Mitglied in einer Active Directory Domäne und nutzt Kerberos für die Authentifizierung.

- `realm = b11b.local` :

Der Realm bezieht sich auf Kerberos. Üblicherweise entspricht der Realm dem Domänenname im Active Directory.

- `password server = 10.0.142.2 :`
Hierrüber wird die IP Adresse des Domänen Controllers angegeben, welcher für die Authentifizierung von Benutzern verantwortlich ist
- `workgroup = B11B :`
Mit diesem Attribut wird die Arbeitsgruppe spezifiziert. Die Arbeitsgruppe spiegelt in der Regel der Active Directory Domäne.
- `kerberos method = secrets and keytab :`
Diese Option legt fest, dass Samba sowohl das traditionelle Geheimnis (`secrets.tdb`) als auch eine Keytab-Datei verwendet, um die Kerberos Authentifizierung durchzuführen. Die Keytab-Datei enthält die Kerberos-Schlüssel für den Server, die zur Authentifizierung benötigt werden.
- `winbind use default domain = yes :`
Diese Einstellung bewirkt, dass `winbind` Benutzernamen und Gruppennamen ohne den Domänenpräfix (`DOMAIN\user`) zurückgibt, wenn die Anfragen aus der Standarddomäne stammen. Dies vereinfacht die Verwendung von Benutzernamen in Befehlen und Skripten.
- `winbind refresh tickets = yes :`
Diese Option sorgt dafür, dass `winbind` Tickets von Kerberos automatisch erneuert, bevor sie ablaufen. Dies ist wichtig für kontinuierlichen Zugriff ohne Unterbrechungen.
- `winbind enum users = yes` und `winbind enum groups = yes :`
Diese Einstellungen erlauben die Auflistung von Benutzern und Gruppen durch `winbind`. Das bedeutet, dass Befehle wie `getent passwd` oder `getent group` alle Benutzer und Gruppen aus dem Active Directory anzeigen können.
- `template shell = /bin/bash :`
Gibt die Standardshell an, die neuen Benutzern zugewiesen wird, die über `winbind` aus dem Active Directory abgerufen werden. Hier ist die Standardshell `/bin/bash`.
- `idmap config * : range = 10000 - 19999 :`
Diese Einstellung definiert einen Bereich für die Zuordnung von IDs (UIDs und GIDs) für Benutzer und Gruppen, die nicht aus der spezifischen Domäne kommen. Hier wird der Bereich von 10000 bis 19999 verwendet.
- `idmap config B11B : backend = rid :`
Diese Option legt fest, dass für die Domäne B11B das `rid` Backend verwendet wird. Das `rid` Backend verwendet die relative ID (RID) des Windows-Sicherheitskennzeichens (SID), um UID und GID Werte zu generieren. Dies ermöglicht eine konsistente Zuordnung von Benutzer- und Gruppen-IDs zwischen Windows und Unix/Linux.
- `idmap config B11B : range = 100000 - 199999 :`
Diese Einstellung legt den Bereich für Benutzer und Gruppen aus der Domäne B11B fest. Hier wird der Bereich von 100000 bis 199999 verwendet. Dies trennt die IDs von denen, die für andere Domänen oder den lokalen Bereich verwendet werden.
- `inherit acls = Yes :`
Diese Option stellt sicher, dass Access Control Lists (ACLs), die auf Dateien und Verzeichnissen im Dateisystem gesetzt sind, von Samba beachtet und vererbt werden. Das bedeutet, dass die Zugriffsrechte, die in einem Verzeichnis definiert sind, auch für die darin enthaltenen Dateien und Unterverzeichnisse gelten.

Anpassung der `/etc/resolv.conf`

Damit der Domänenbeitritt funktioniert, muss sichergestellt werden, dass der DNS Server des Domänen Controllers in der Datei `/etc/resolv.conf` eingetragen ist:

```
search b11b.local
nameserver 10.0.142.2
```

TERMINAL

Hinzufügen des Rechners zur Domäne

Sobald diese Konfiguration erstellt wurde, muss der Rechner in die Active Directory Domäne aufgenommen werden:

```
net ads join -U admin
```

TERMINAL

- **net :**
net ist ein spezieller Samba Befehl, der im Zusammenhang mit Netzwerkdiensten verwendet wird.
- **ads :**
ads ist ein spezieller Befehl, der darauf hinweist, dass man im Active Directory Kontext Aktionen ausführt.
- **join :**
Mit Hilfe von join wird der entsprechende Rechner zu der Domäne innerhalb der Samba Konfiguration hinzugefügt.
- **-U :**
U steht für User. Hierrüber wird der Benutzer spezifiziert, welcher für das Hinzufügen des Rechners verantwortlich ist.

Prüfen, ob Rechner zur Domäne hinzugefügt wurde

```
net ads testjoin
```

TERMINAL

- **testjoin :**
Der Befehl testjoin prüft, ob ein Rechner erfolgreich zur Domäne hinzugefügt wurde. Wenn der Befehl erfolgreich ausgeführt wird, bedeutet dies, dass die Verbindung zur Domäne korrekt hergestellt wurde und der Computer Teil der Domäne ist.

Status der Verbindung zur Active Directory anzeigen

```
net ads -P status
```

TERMINAL

- **-P :**
Mit diesem Parameter wird das gespeicherte Ticket verwendet um eine Authentifizierung durchzuführen.
- **status :**
Gibt den aktuellen Status der Verbindung zur Active Directory Domäne aus. Es wird folglich geprüft, ob der entsprechende Computer weiterhin ordentlich authentifiziert und somit verbunden ist.

Infos zur Active Directory Domäne abrufen

```
net ads info
```

TERMINAL

Mit info können allgemeine Informationen zur Domäne abgerufen werden. Informationen sind z. B. der Name der Domäne, der Name des Domänen Controllers, der Kerberos Realm etc.

Verbindung testen

Bevor die Verbindung wirklich auf deren Funktionalität getestet wird, sollte man einige Dienste neustarten:

Samba neustarten

```
systemctl restart smbd
```

TERMINAL

Samba NetBIOS neustarten

```
systemctl restart nmbd
```

TERMINAL

winbind neustarten

```
systemctl restart winbind
```

TERMINAL

Sind alle Dienste neugestartet kann mit Hilfe des Befehls `wbinfo` generell Infos aus der Domäne abgerufen werden:

Benutzer aus der Domäne abrufen

```
wbinfo -u
```

TERMINAL

Gruppen aus der Domäne abrufen

```
wbinfo -g
```

TERMINAL

Lokale und Active Directory Benutzer anzeigen

```
getent passwd
```

TERMINAL

Zusätzliche Konfigurationen

PAM

Auch in Bezug auf `winbind` wird PAM erneut während der Installation angepasst. Betroffen sind dabei nachfolgende Komponenten:

`/etc/pam.d/common-account`

Die `common-account` ist zuständig für den Authentifizierungsprozess. Es wird geprüft, ob ein Account überhaupt zugelassen oder gar eingeschränkt ist. Innerhalb des Moduls wird eine Zeile für `winbind` benötigt:

```
...
account [success=2 new_authtok_reqd=done default=ignore] pam_unix.so
account [success=1 new_authtok_reqd=done default=ignore] pam_winbind.so
...
```

TERMINAL

Im ersten Fall wird folglich mit Hilfe des `pam_unix.so` geprüft, ob ein Account vorhanden ist. Sollte das allerdings nicht der Fall sein, so greift das Modul für `winbind`.

`/etc/pam.d/common-auth`

Auch in Bezug auf die Authentifizierung wird in `winbind` Modul hinzugefügt:

```
...
auth [success=1 default=ignore] pam_winbind.so krb5_auth krb5_ccache_type=FILE cached_login try_first_pass
...
```

TERMINAL

`/etc/pam.d/common-session`

Das Modul wird wieder benötigt, damit entsprechende Aktionen beim Start und Beenden einer Benutzersitzung ausgeführt werden. Möchte man, bei einer Anmeldung mit einem Benutzer aus dem Active Directory auch wieder ein Home Verzeichnis bekommt, so muss nachfolgende Zeile in die `/etc/pam.d/common-session` hinzugefügt werden:

```
...
session required pam_mkhomedir.so skel=/etc/skel/ umask=0022
...
```

TERMINAL